LIVE DEMO RUNBOOK

VerdeBase — DBMS Viva Walkthrough

A 60-minute script for presenting the VerdeBase (PlantCare) PostgreSQL project: the live app first, then DBMS Studio, then pgAdmin as independent proof. Click a phase below to jump to it.

⏱ 60 min total | app + DBMS Studio + pgAdmin4 | PostgreSQL · Next.js

---

before — Pre-flight

0:00–0:05 — Introduction

0:05–0:20 — App walkthrough

0:20–0:50 — DBMS Studio

0:50–0:57 — pgAdmin proof

0:57–1:00 — Wrap-up & Q&A

Pre-flight checklist

DO THIS 10 MIN BEFORE SHE ARRIVES

Tick these off once, quietly, before your professor sits down. Nothing here should happen while she's watching.

- [ ] PostgreSQL service running
- [ ] Dev server up: npm run dev → localhost:3000
- [ ] pgAdmin4 open, connected to verdebase
- [ ] Logged out of any stale session (clear cookies if unsure)
- [ ] Browser zoom reset to 100%
- [ ] Window arranged: app + pgAdmin both visible
- [ ] Know both logins by heart (table below)
- [ ] Chatbot page skipped unless a Groq API key is set

ROLE	EMAIL	PASSWORD
Admin (DBMS Studio)	admin@verdebase.com	Admin@123
Regular user	jane@verdebase.com	User@123

pgAdmin connection localhost : 5432 · db **verdebase** user **postgres**

0:00 – 0:05 Introduction

5 MIN

Set the frame before touching the keyboard: one app, one real PostgreSQL database, two windows open on it at once.

1. Arrange your screen: `localhost:3000` (the app) on one side, pgAdmin4 open to `verdebase → Schemas → public` on the other — or on a second monitor if you have one.

   > **SAY** *"This is VerdeBase, a plant-care manager. It's a Next.js app, but every piece of data you'll see lives in this PostgreSQL database — nothing is mocked or hardcoded."*

2. Name the scale up front: 14 tables, 6 views, 10+ stored functions, 5 triggers, 49 indexes.

   > **SAY** *"The schema is normalized to 3NF — for example species data and care-profile data are deliberately split into two tables so updating a care interval never touches the species row."*

3. Preview the plan: "I'll show the app as a normal user first, then switch to an admin view called DBMS Studio that's a live window into the catalog itself, then cross-check everything in pgAdmin."

0:05 – 0:20 Core app, as a regular user

15 MIN · JANE@VERDEBASE.COM

Log in as Jane (role_id 2 — a normal user, no DBMS Studio access) and show real CRUD against real tables, ending with a trigger firing live.

1. Log in at `/auth` as **jane@verdebase.com**. Land on the Dashboard — point out the stat tiles (Total/Healthy/Needs Attention/Critical) and Recent Activity feed.

   `Aggregation query —user_dashboard_statsview`

2. Go to **My Plants** → **Add Plant**. Fill a name and location, leave Species as "— None —", submit.

   > **SAY** *"Species is an optional foreign key — `species_id` is nullable, so a plant can exist before you've catalogued what it is. Watch it appear instantly."*

   `Nullable FK · ON DELETE SET NULL`

3. Open that new plant (**View**) and switch to pgAdmin for a second — run `SELECT * FROM plants ORDER BY plant_id DESC LIMIT 1;` in the Query Tool to show the exact same row.

4. Back in the app, on an existing plant (e.g. **My Monstera**), open the **Diseases** tab → **Add Disease Log** → pick *Root Rot* or *Leaf Spot* → Save.

   > **SAY** *"Watch the health badge at the top — I didn't touch it. A database trigger just recalculated the plant's health status and wrote an audit log entry and a notification, automatically."*

   `AFTER INSERT trigger → cascading UPDATE + INSERT`

5. Quick tour: **Species** (3NF — explain the separate `care_profiles` table), **Reminders** (shows a `TEXT[]` array column for schedule days), **Analytics** (window function + date-trend queries).

0:20 – 0:50 DBMS Studio, as admin

30 MIN · ADMIN@VERDEBASE.COM

Sign out, log back in as **admin@verdebase.com** (role_id 1), click the **DBMS Studio** toggle top-left. This is the core of the viva — it reads PostgreSQL's own system catalogs (`information_schema`, `pg_stat_user_tables`, `pg_indexes`) live, nothing is precomputed.

1. **Database Overview** — point at the live counts (14 tables / 6 views / 48 functions / 10 triggers / 49 indexes) and the per-table row/size breakdown.

   `pg_stat_user_tables · pg_total_relation_size`

2. **ER Explorer** — the full diagram, generated from `information_schema` on page load. Click `plants` to highlight its PK (gold) and FKs (orange) and the relationship lines to `users` / `species`.

   `ER model · referential integrity`

3. **Relational Model** & **Normalization** pages — use these if she asks "why is care_profiles separate from species?" The normalization page walks UNF → 1NF → 2NF → 3NF with this exact schema.

4. **Tables** → click into `plants` → show columns, the `PRIMARY KEY` / `FOREIGN KEY` badges, the index list, and 5 live sample rows.

5. **Constraints** page — walk through the constraint types shown (PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE, NOT NULL) each mapped to a real column, e.g. `plants_user_nickname_unique UNIQUE(user_id, nickname)` and the health_status CHECK list.

6. **SQL Lab** — run three of the built-in examples back to back (click a query in the sidebar, then **Run Query**):

then **Run Query**):

Health Status Distribution

`GROUP BY + window fn`

`SUM(COUNT(*)) OVER ()` computes each group's % of the whole in one pass.

Plants with Active Diseases

`correlated subquery`

Outer query filtered by an `IN (SELECT …)` against `disease_records`.

Get Plant Statistics

`LATERAL + SRF`

Calls the `get_plant_statistics()` set-returning function per row via `CROSS JOIN LATERAL`.

Users with Critical Plants

`JOIN + HAVING`

Multi-table join aggregated, then filtered post-aggregation with HAVING.

7 **Views** — open `plant_health_summary` and `species_care_overview`, show their definitions and run them. Tie back: "this is literally what powers the My Plants page."

8 **Functions** — open `calculate_plant_health_score(plant_id)`, read the PL/pgSQL body (deducts points for overdue watering, active disease, blends in the manual health score), then run it live from the SQL Lab: `SELECT calculate_plant_health_score(1);`

9 **Triggers** — show the trigger-flow diagram, then point back at the disease-log demo from the app section: that's this exact trigger firing.

10 **Transactions** page — explain ACID using the four cards (Atomicity/Consistency/Isolation/Durability), each mapped to a real VerdeBase scenario already on screen.

11 **Query Performance** — paste the "Indexed Plant Lookup" example, click **EXPLAIN ANALYZE (Live Run)**. Point at the plan node: Index Scan, with real cost/timing numbers, not estimates.

`Cost-based optimizer · B-tree index`

12 **Audit Logs** — scroll to the top entries, find the `disease_records` INSERT from your earlier demo and the LOGIN events from this session. "Every mutation is traced, written by triggers, not application code — so it can't be bypassed."

IF A PAGE LOOKS EMPTY OR ERRORS

Hit refresh once — Next.js dev builds occasionally hiccup on first compile of a route. Everything here re-queries Postgres on load, so a refresh always recovers. Worst case, fall back to pgAdmin for that one data point.

7 MIN

0:50 – 0:57 pgAdmin4 — independent proof

7 MIN

The point of this block: prove DBMS Studio isn't a simulation, and prove constraints live in the database, not just in the UI.

1. In pgAdmin's Query Tool, run `SELECT * FROM species_care_overview;` — same rows, same numbers, as the Species page.
2. Try to break a constraint on purpose: `INSERT INTO plants (user_id, plant_name, health_status) VALUES (2, 'Test', 'on_fire');`

   > **SAY** *"That fails with a CHECK constraint violation — `health_status` can only be one of four values, enforced by Postgres itself, independent of whatever the app's form allows."*

   `CHECK constraint rejection —expected to error, that's the demo`
3. Right-click `plants` → Properties → Constraints tab — show the real constraint list (PK, 2 FKs, the UNIQUE on `(user_id, nickname)`, 2 CHECKs) as pgAdmin sees them, matching the app's Constraints page exactly.

0:57 – 1:00 Wrap-up & anticipated questions

BUFFER

Leave the last few minutes open. Here's quick ammunition for the questions that come up most.

- **Why integer primary keys instead of UUID?** +
- **Why split species and care_profiles into two tables?** +
- **What's the difference between a view and a function here?** +
- **What happens on a cascading delete?** +
- **Is DBMS Studio just showing static/fake data?** +